# Inception-of-Things (IoT): Learning Guide and Team Task Plan

Sep 30, 2026 · @Redouane

## Overview

Inception-of-Things is a System Administration project that introduces Kubernetes in three ordered parts: K3s with Vagrant (P1), K3s with three web apps and an Ingress (P2), and K3d with Argo CD for GitOps (P3), plus an optional GitLab bonus. It must be done entirely inside a virtual machine, and only what is in your Git repository is evaluated, on your group's computer.

This doc has two parts: **Part A** covers what you need to learn, and **Part B** is the task plan for you and your teammate.

## Part A: Learning

Both teammates should understand every concept below, because evaluators can ask either of you about any part of the project.

### Concepts to understand

- **Kubernetes (K8s) basics:** how a cluster works, and the difference between a controller (server) node and an agent (worker) node.
- **K3s vs K3d:** K3s is lightweight Kubernetes that runs directly on a machine or VM. K3d runs K3s inside Docker containers. The subject explicitly says you must understand this difference.
- **Workloads:** Pods, Deployments, ReplicaSets, and Services, including running 3 replicas of one app.
- **Ingress:** routing traffic to different apps based on the HTTP `Host` header. K3s ships with Traefik as its Ingress controller.
- **Namespaces:** isolating resources ( `argocd` , `dev` , and `gitlab` for the bonus).
- **GitOps and continuous delivery:** Git is the source of truth, and Argo CD keeps the cluster in sync with your repository.

- **Docker image tagging:** using versions such as `v1` and `v2` to release and roll out updates.

### Skills to gain

- Writing a modern **Vagrantfile**: box, hostname, private network with static IP, CPU and RAM limits, shell provisioning.
- Writing **shell scripts** to automate installs so setups are repeatable.
- Installing K3s in server mode and agent mode, and joining a worker to a server using its token.
- Using **kubectl** day to day: `get nodes -o wide`, `get all`, `get ns`, `get pods -n dev`, `apply -f`.
- Writing Kubernetes YAML manifests: Deployment, Service, Ingress, and an Argo CD Application.
- Installing Docker, K3d, and Argo CD, and reaching the Argo CD UI.
- Using Git and GitHub, and Docker Hub if you build your own app.
- Network debugging: `ip a`, `ssh`, and `curl -H "Host: app2.com" 192.168.56.110`.
- Reading documentation. The subject explicitly advises reading a lot of it.
- Bonus only: Helm and deploying GitLab on Kubernetes.

### Suggested study order

1. Vagrant and VirtualBox basics, then K3s server and agent install.
2. Pods, Deployments, Services, then Ingress with Traefik.
3. Docker, K3d, and how it differs from K3s.
4. Argo CD and the GitOps flow.
5. Helm and GitLab, only if you attempt the bonus.

## Part B: Tasks (you and your teammate)

The parts must be done in order (P1, then P2, then P3), so the plan below has one lead per part and a review by the other person. Call the two of you **Teammate A** and **Teammate B**, and swap names in as you like.

## Who does what

| Part | Lead | Reviewer | Deliverable |
|---|---|---|---|
| Setup and repo | Teammate A | Teammate B | Git repo with `p1`, `p2`, `p3` folders, each with `scripts/` and `confs/` |
| P1: K3s and Vagrant | Teammate A | Teammate B | `p1/Vagrantfile`, server and worker scripts |
| P2: K3s and three apps | Teammate B | Teammate A | `p2/Vagrantfile`, app manifests, Ingress |
| P3: K3d and Argo CD | Teammate A (install script, K3d, Argo CD) and Teammate B (GitHub repo, app manifests, v1 to v2 test) | Each other | `p3/scripts`, `p3/confs`, public GitHub repo |
| Bonus: GitLab | Decide together, only after P1 to P3 are flawless | Each other | `bonus/` folder |
| Defense prep | Both | Both | Rehearsed demo and explanations |

## Requirements checklist

### General

- [ ] Whole project runs inside a virtual machine
- [ ] Repo root holds `p1`, `p2`, `p3` (and optionally `bonus`), each with `scripts/` and `confs/`
- [ ] Folder and file names double-checked before submission

### P1

- [ ] 2 VMs on the latest stable version of your chosen distro, 1 CPU and 512 MB RAM (or 1024 MB)
- [ ] Names are a team member's login plus `S` and `SW` (example: `wilS`, `wilSW`)
- [ ] IPs are `192.168.56.110` (server) and `192.168.56.111` (worker)
- [ ] Passwordless SSH to both machines
- [ ] K3s in controller mode on the server and agent mode on the worker

- [ ] `kubectl` installed and working
- [ ] Vagrantfile follows modern practices

**P2**

- [ ] 1 VM named login plus `S`, IP `192.168.56.110`, K3s in server mode
- [ ] 3 web apps of your choice
- [ ] `app1.com` shows app1, `app2.com` shows app2, any other host shows app3
- [ ] App 2 runs 3 replicas
- [ ] Ingress ready to show the evaluators

**P3**

- [ ] No Vagrant; K3d installed on the VM
- [ ] Script that installs Docker, K3d and all other tools, ready to run during the defense
- [ ] Two namespaces: one for Argo CD and one named `dev`
- [ ] Public GitHub repo with a team member's login in its name
- [ ] App has two versions tagged `v1` and `v2` (Wil's `wil42/playground` on port 8888, or your own public Docker Hub image with visible differences)
- [ ] Argo CD deploys the app to `dev` from GitHub automatically
- [ ] Live demo: change v1 to v2 in GitHub and show the app update

**Bonus (graded only if the mandatory part is flawless)**

- [ ] Latest GitLab running locally in a namespace named `gitlab`
- [ ] Configured to work with the cluster
- [ ] Everything from P3 works with local GitLab
- [ ] All files in `bonus/`

## Step-by-step plan

### Step 0: Preparation (both)

1. Choose the host VM tool, the Vagrant provider (for example VirtualBox), and the Linux distro.
2. Pick whose login is used for naming (example: `wilS`, `wilSW`).
3. Create the Git repo with the folder structure above and give both of you access.
4. Agree on a branch and commit routine so you do not overwrite each other's work.

### Step 1: P1, K3s and Vagrant (lead: Teammate A)

1. Write `p1/Vagrantfile` with two VMs, hostnames, private-network IPs `.110` and `.111`, and the CPU and RAM limits.
2. Write a server script: install K3s in controller mode, bind it to the `192.168.56.x` interface, and save the node token.
3. Write a worker script: install K3s in agent mode using the server IP and the token.
4. Install `kubectl` on the server.
5. Run `vagrant up`, then check `kubectl get nodes -o wide`: both nodes should be `Ready` with internal IPs `.110` and `.111`.
6. Check the network with `ip a`. The subject's `eth0` and `eth1` names are outdated, so use your real interface names.
7. Teammate B reviews, rebuilds from scratch once, and confirms it works.

### Step 2: P2, K3s and three apps (lead: Teammate B)

1. Write `p2/Vagrantfile` for one VM with IP `192.168.56.110` and K3s in server mode.
2. In `confs/`, write a Deployment and a Service for each app, with `replicas: 3` on app 2.
3. Write an Ingress using Traefik: host `app1.com` to app1, host `app2.com` to app2, and a default backend to app3.
4. Apply everything with `kubectl apply -f`.
5. Test with `curl -H "Host: app1.com" 192.168.56.110`, then `app2.com`, then a different or missing host (should show app3).
6. Be ready to show `kubectl get all` and `kubectl get ingress`.
7. Teammate A reviews and rebuilds from scratch once.

### Step 3: P3, K3d and Argo CD (split between both)

1. **Teammate A:** write `p3/scripts` install script for Docker, `kubectl`, K3d and any other tools, then create the K3d cluster.
2. **Teammate A:** create the `argocd` and `dev` namespaces and install Argo CD into `argocd`. Get the admin password and reach the UI.
3. **Teammate B:** create the public GitHub repo (login in the name) with `deployment.yaml` and a service using `wil42/playground:v1` on port 8888, or your own app.
4. **Teammate B (only if building your own app):** push a public Docker Hub image tagged `v1` and `v2`, with visible differences between them.

5. **Teammate A:** write the Argo CD Application manifest in `p3/confs` pointing at the GitHub repo, destination namespace `dev`, automated sync.
6. **Both:** verify with `kubectl get ns`, `kubectl get pods -n dev`, and `curl http://localhost:8888/` returning `v1`.
7. **Both:** change `v1` to `v2` in `deployment.yaml`, commit and push, wait for Argo CD to sync, and `curl` again to confirm `v2`.

### Step 4: Bonus (optional, decide together)

1. Only start once P1 to P3 are flawless.
2. Create the `gitlab` namespace and deploy the latest GitLab locally, for example with Helm.
3. Configure GitLab to work with the cluster.
4. Rework P3 so Argo CD pulls from your local GitLab repo.
5. Put everything in `bonus/`.

### Step 5: Submission and defense prep (both)

1. Double-check folder and file names, then push everything to the repo.
2. Each person rehearses explaining: K3s vs K3d, Ingress, namespaces, and the GitOps flow.
3. Run the P3 install script live on a clean VM at least once.
4. Practice the v1 to v2 demo until either of you can do it alone.
5. Remember the evaluation happens on your group's computer, so test there.